



Test Automation Framework (TAF::Perl)

Sysbench-LUA + BMK (Benchmark Kit) Quick Start

MariaDB Foundation <https://mariadb.org>

Open, vendor-neutral, community-driven tooling for database testing and benchmarking.

TAF-Perl is maintained and published by the MariaDB Foundation to support transparent, reproducible, and fair performance evaluation across the open source database ecosystem. The framework is part of the Foundation's mission to provide open tooling that benefits the entire database community.

© MariaDB Foundation. All rights reserved.

Sysbench-Lua Overview

Sysbench is an open workload framework created and maintained by **Alexey Kopytov**. It provides a collection of OLTP-focused Lua workloads designed to evaluate transactional performance characteristics across database engines. Sysbench is documented at:

<https://github.com/akopytov/sysbench>

Within TAF, the sysbench-lua test suite provides a deterministic and contributor-proof method for running transactional OLTP workloads across supported database engines. TAF does not modify sysbench, does not validate benchmark correctness, and does not certify results. The suite focuses on lifecycle control, configuration merging, and consistent execution across environments.

The test suite supports the upstream OLTP Lua workloads, including read-write, read-only, point-select, update-index, update-non-index, and related variants. Only the database plugin components are exercised; system-level tests such as fileio, memory, threads, and CPU are intentionally excluded.

Sysbench-lua must be compiled before use. TAF provides build actions such as **build-client**, **build-client-run-tests**, and **init-start-db-build-client-run-tests** to ensure a consistent and reproducible client environment.

The suite is responsible for initialization, deterministic merging of default properties, user properties, and command-line overrides, execution of supported test cases, and collection of results for downstream reporting. It enforces stable and reproducible behavior across repeated runs.

The suite does not implement sysbench itself, does not manage database provisioning, and does not interpret or certify benchmark results. All workload semantics originate from upstream sysbench, and all OLTP concepts originate from the publicly available sysbench documentation.

By integrating sysbench-lua into TAF, the goal is to provide a consistent and script-driven method for running transactional workloads across MariaDB and MySQL. PostgreSQL and additional engines will be evaluated after the beta release.

BMK Overview

BMK is a workload framework created and maintained by **Dimitri Kravtchuk**. It extends the traditional sysbench OLTP model with additional range-based, index-focused, and point-select-driven test cases designed to expose performance characteristics not covered by standard sysbench workloads. BMK is documented at: <http://dimitrik.free.fr/BMK/>

Within TAF, the BMK test suite provides a deterministic and contributor-proof method for running extended OLTP workloads across supported database engines. TAF does not modify BMK, does not validate benchmark correctness, and does not certify results. The suite focuses on lifecycle control, configuration merging, and consistent execution across environments.

The suite supports the regular sysbench OLTP workloads as well as the BMK-specific extensions, including update-range, update-index-range, non-index-range, point-select-range, and the `SELECT_*` range-based analytical micro-benchmarks. These workloads provide deeper visibility into index behavior, range access patterns, and mixed read/write contention scenarios.

TAF runs BMK unmodified. All workload semantics originate from BMK itself, and all OLTP and range-based concepts originate from the publicly available BMK documentation.

The suite is responsible for initialization, deterministic merging of default properties, user properties, and command-line overrides, execution of the supported test cases, and collection of results for

downstream reporting. It enforces stable and reproducible behavior across repeated runs.

The suite does not implement BMK itself, does not manage database provisioning, and does not interpret or certify benchmark results.

By integrating BMK into TAF, the goal is to provide a consistent and script-driven method for running extended OLTP and range-based workloads across database makers.

TAF - Test Suite Structure and Locations

The sysbench-lua test suite follows the standard TAF layout. All components are located under the TAF installation root.

Test Suite Module: taf-perl/test_suites/sysbench-lua.pm

This file defines lifecycle routines, configuration handling, and execution flow.

Default Properties: taf-perl/properties/default/sysbench_lua_default.properties

Provides deterministic defaults for all supported databases. Users must not modify this file.

Sysbench-lua Client Source: taf-perl/client_source/sysbench-lua/

Contains the upstream sysbench-lua source code used by the test suite. Sysbench-lua must be compiled before use. TAF provides actions such as:

build-client

Build the test suite client.

build-client-run-tests

Build the client and run tests.

install-init-start-db-build-client-run-tests

Install the database, initialize it, start it, build the client, run tests.

init-start-db-build-client-run-tests

Initialize and start the database, build the client, run tests.

BMK Client Source: taf-perl/client_source/BMK/

Contains BMK used by the test suite.

Configuration Resolution Order

TAF resolves configuration in a strict and deterministic order. This guarantees that every test suite behaves the same way across environments and that users always know which value is in effect. All test suites, including sysbench-lua, follow the same precedence rules.

1. TAF and sysbench-lua Default Files

Both TAF and the sysbench-lua test suite provide default properties files. These files define safe, stable baseline settings and may be overridden, but must never be modified directly. Users override defaults only through user properties files.

2. Auto-Populated Values

When certain settings are left unset, TAF and the sysbench-lua test suite automatically populate them using installation paths or other environment-derived values. These values apply only when the user has not provided an explicit override.

3. User Properties Files

Users supply a test-case-specific properties file to override defaults. This is the intended method for running TAF. Any key defined in a user properties file replaces both the default value and any auto-populated value.

4. Command-Line Overrides

Command-line options have the highest precedence. Any key specified on the command line replaces the value from the user properties file, the auto-populated value, and the default.

This resolution order ensures predictable, contributor-proof behavior. The final configuration used by the test suite is always the result of applying these four layers in order, with command-line options taking priority over all other sources.

Test Suite Properties

Test suite properties begin with the prefix: **sysbench_lua.<option>=**

These properties control workload behavior, client paths, agent settings, and database specific extensions. Defaults come from the test suite's default properties file. Users may override any value through a user properties file or a command-line option.

Examples:

- sysbench_lua.number_of_tables=8
- sysbench_lua.number_of_rows=10000
- sysbench_lua.db_driver= mariadb
- sysbench_lua.use_bmk=true
- sysbench_lua.intermediate_result=2
- sysbench_lua.debug_sysbench= true

TAF Options That Affect Test Suite Execution

The following is a sample of TAF options and properties that impact test suite execution. These options directly influence how the HammerDB TPROC-H test suite runs. Each option has both a command-line options form (O) and a properties form (P).

This is not a complete list. TAF provides many additional options and properties, all of which can be viewed by running:

perl taf.pl --help

Users are encouraged to review the full usage output to understand the complete set of capabilities.

Examples.

Action

O = --action=<action>

P = taf.action=<action>

Controls the overall TAF workflow.

Duration

O = --duration=<seconds|min>

P = taf.duration=<seconds|min>

How long to run each test.

Seconds vs. minutes depends entirely on the test suite implementation. (sysbench-lua is sec)

Default: 30

Iterations

O = --iterations=<number>

P = taf.iterations=<number>

Number of iterations to run for each thread count.

Default: 1

Thread Counts

O = --threads=1

P = taf.threads=1

Defines the list of thread counts to run.

These options combine with the test suite's own properties to determine the final workload behavior. All values are resolved using TAF's deterministic precedence rules: defaults, auto-populated values, user properties, then command-line overrides.

Sysbench-LUA Help

The following is example of running test suite help, and the output of the test suite's help.

perl taf.pl --test-suite=sysbench-lua --list-test-suites-help

TAF TSM::LoadTestSuite -> : Called

TAF TSM::LoadTestSuite -> : Attempting to load: sysbench-lua

TAF TSM::LoadTestSuite -> : Test suite path : /home/taf-perl/test_suites/sysbench-lua.pm

TAF TSM::LoadTestSuite -> : Test suite has completed loading.

TAF TSM::LoadTestSuite -> : Complete

Sysbench OLTP Test Suite HELP

Workload Overview:

Sysbench is a Lua-driven microbenchmark tool used to evaluate database performance. The OLTP tests simulate simple transactional workloads using a synthetic schema. These tests are not TPC-C or TPROC-C compliant; they are lightweight, repeatable stress tests. TAF wraps Sysbench to provide deterministic configuration and consistent result capture across environments.

Key Concepts:

threads : Number of concurrent client threads.
time : Duration of the run phase in seconds.
events : Total number of events to execute. Overrides time.
rate : Target events per second. 0 means unlimited.
warmup_time : Pre-run warmup period in seconds.
report_interval : Interval for intermediate statistics.
db_driver : Database driver (mysql, pgsql, etc.).
db_ps_mode : Prepared statement mode (auto, disable, force).
validate : true/false. Enables result validation.
histogram : true/false. Enables latency histogram.
percentile : Percentile to report when histogram=true.

Important Invariants:

threads must be ≥ 1 .

time must be ≥ 1 when events is not set.
events must be ≥ 1 when time is not used.
rate must be ≥ 0 .
warmup_time must be ≥ 0 .
report_interval must be ≥ 0 .
percentile must be between 1 and 100 when histogram=true.
db_ps_mode must be one of: auto, disable, force.

OLTP Test Types:

oltp_read_only : Read-only point selects.
oltp_read_write : Mixed read/write workload.
oltp_update_index : Updates on indexed columns.
oltp_update_non_index : Updates on non-indexed columns.
oltp_insert : Insert-only workload.
oltp_delete : Delete-only workload.
oltp_point_select : Simple point-select queries.

=====

Properties/Test Matchup

=====

The following helps match Sysbench Lua properties to OLTP tests.

By changing these values, users can modify how many operations are included in each request.

Example: To run 3 simple-range queries per request instead of 1,
set: sysbench_lua.oltp_simple_ranges=3

POINT_SELECT_MODIFIABLE:

sysbench_lua.oltp_point_selects

SELECT_SUM_RANGES_MODIFIABLE:

sysbench_lua.oltp_sum_ranges

SELECT_ORDER_RANGES_MODIFIABLE:

sysbench_lua.oltp_order_ranges

SELECT_DISTINCT_RANGES_MODIFIABLE:

sysbench_lua.oltp_distinct_ranges

OLTP_RW_MODIFIABLE:

sysbench_lua.oltp_simple_ranges

sysbench_lua.oltp_sum_ranges

sysbench_lua.oltp_order_ranges

sysbench_lua.oltp_point_selects
sysbench_lua.oltp_distinct_ranges
sysbench_lua.oltp_index_updates
sysbench_lua.oltp_non_index_updates
sysbench_lua.oltp_del_ins

OLTP_RO_MODIFIABLE:

sysbench_lua.oltp_simple_ranges
sysbench_lua.oltp_sum_ranges
sysbench_lua.oltp_order_ranges
sysbench_lua.oltp_point_selects
sysbench_lua.oltp_distinct_ranges

OLTP_RW_PS_ONLY_MODIFIABLE:

sysbench_lua.oltp_point_selects
sysbench_lua.oltp_index_updates
sysbench_lua.oltp_non_index_updates
sysbench_lua.oltp_del_ins

OLTP_WO_MODIFIABLE:

sysbench_lua.oltp_index_updates
sysbench_lua.oltp_non_index_updates
sysbench_lua.oltp_del_ins

UPDATE_KEY_TRANSACTIONAL_MODIFIABLE:

sysbench_lua.oltp_index_updates

UPDATE_NO_KEY_TRANSACTIONAL_MODIFIABLE:

sysbench_lua.oltp_non_index_updates

UPDATE_KEY_NO_KEY_INT_TRANSACTIONAL_MODIFIABLE:

sysbench_lua.oltp_index_non_index_updates

Default Tests:

OLTP_RO
OLTP_RW
UPDATE_KEY
UPDATE_NO_KEY

POINT_SELECT
SELECT_SIMPLE_RANGES
SELECT_SUM_RANGES
SELECT_ORDER_RANGES
SELECT_DISTINCT_RANGES

Legal Tests:

OLTP_RO
OLTP_RW
UPDATE_KEY
UPDATE_NO_KEY
POINT_SELECT
SELECT_SIMPLE_RANGES
SELECT_SUM_RANGES
SELECT_ORDER_RANGES
SELECT_DISTINCT_RANGES
DELETE
INSERT
OLTP_INSERT_INT
OLTP_RO_MODIFIABLE
OLTP_RW_MODIFIABLE
OLTP_WO_MODIFIABLE
OLTP_RW_PS_ONLY_MODIFIABLE
UPDATE_KEY_TRANSACTIONAL_MODIFIABLE
UPDATE_NO_KEY_TRANSACTIONAL_MODIFIABLE
UPDATE_KEY_NO_KEY_INT_TRANSACTIONAL_MODIFIABLE
POINT_SELECT_MODIFIABLE
SELECT_DISTINCT_RANGES_MODIFIABLE
SELECT_ORDER_RANGES_MODIFIABLE
SELECT_SIMPLE_RANGES_MODIFIABLE
SELECT_SUM_RANGES_MODIFIABLE
PARSER
PARSER-RO
BMK_RW_UPDATE_RANGE
BMK_WO_UPDATE_RANGE
BMK_RW_UPDATE_INDEX_RANGE

BMK_RW_UPDATE_NON_INDEX_RANGE
BMK_RW_PS_UPDATE_RANGE
BMK_RW_PS_UPDATE_INDEX_RANGE
BMK_RW_PS_UPDATE_NON_INDEX_RANGE
CONNECT
TPCC

Resolved Properties:

sysbench_lua.args = not defined
sysbench_lua.auto_inc = on
sysbench_lua.bmk_archive = external_tools/BMK-kit.tgz
sysbench_lua.bmk_by_sec_index = 0
sysbench_lua.bmk_check_character_set = not defined
sysbench_lua.bmk_exe = client_source/BMK/sysbench-1.1-mysql80-ssl111L-x64
sysbench_lua.bmk_install = client_source/BMK/
sysbench_lua.bmk_libs = client_source/BMK/lib-ssl111L-x64
sysbench_lua.bmk_lua_scripts_dir = client_source/BMK/sb_exec/lua
sysbench_lua.bmk_mysql_ssl = not defined
sysbench_lua.bmk_partitions = not defined
sysbench_lua.bmk_prepare_threads = 8
sysbench_lua.bmk_reconnect = not defined
sysbench_lua.bmk_source = ./client_source/BMK/sb_exec/lua
sysbench_lua.bmk_update_range_size = 0
sysbench_lua.cmake_args = -DWITH_MYSQL=on -DCMAKE_BUILD_TYPE=Release
sysbench_lua.connector = C-API
sysbench_lua.db_driver = mariadb
sysbench_lua.db_ps_mode = auto
sysbench_lua.debug_full_sysbench = not defined
sysbench_lua.debug_sysbench = not defined
sysbench_lua.def_duration = 300
sysbench_lua.def_engine = INNODB
sysbench_lua.def_myisam_rows = not defined
sysbench_lua.def_rows = 1000000
sysbench_lua.def_threads = 8,16,32,64,128,246,512,1024
sysbench_lua.error_log = not defined
sysbench_lua.exe = client_source/sysbench-lua/sysbench
sysbench_lua.forced_shutdown = 1

sysbench_lua.forced_shutdown_sec = 0
sysbench_lua.intermediate_result = not defined
sysbench_lua.load_args = not defined
sysbench_lua.lua_scripts_dir = client_source/sysbench-lua/src/lua
sysbench_lua.mysql_character_set = not defined
sysbench_lua.number_of_partitions = not defined
sysbench_lua.number_of_rows = 10000
sysbench_lua.number_of_tables = 1
sysbench_lua.oltp_del_ins = 1
sysbench_lua.oltp_dist_type = uniform
sysbench_lua.oltp_distinct_ranges = 1
sysbench_lua.oltp_index_updates = 1
sysbench_lua.oltp_lua_script = oltp_read_only.lua
sysbench_lua.oltp_non_index_updates = 1
sysbench_lua.oltp_order_ranges = 1
sysbench_lua.oltp_point_selects = 10
sysbench_lua.oltp_simple_ranges = 1
sysbench_lua.oltp_skip_trx = on
sysbench_lua.oltp_sum_ranges = 1
sysbench_lua.parser = not defined
sysbench_lua.pre_test_done = 0
sysbench_lua.range_size = not defined
sysbench_lua.seed_rng = not defined
sysbench_lua.self = SYSBENCH-LUA
sysbench_lua.source = client_source/sysbench-lua/
sysbench_lua.test_args = not defined
sysbench_lua.test_client_version = 1.1.0
sysbench_lua.thread_init_timeout = 0
sysbench_lua.use_bmk = 0

=====

===== Helpful Sites =====

=====

Sysbench GitHub: <https://github.com/akopytov/sysbench> (Alexey Kopytov)

BMK: <http://dimitrik.free.fr/BMK/> (Dimitri Kravtchuk)

<https://www.mariadb.org>

=====

===== End Sysbench Help =====

=====

Sysbench-LUA Usage Command Example

Basic command-line for executing the provided example MariaDB user's properties-file.

perl taf.pl --prop=./properties/mariadb/beta/sysbench_lua.properties

User Properties File: properties/mariadb/beta/sysbench_lua.properties

```
#####  
# TAF SETTINGS (GLOBAL)  
#  
# These settings control TAF behavior for this sysbench-lua or BMK test run:  
# - what action to perform  
# - which test suite to execute  
# - how long to run the workload  
# - how many iterations and threads to use  
# - warmup behavior  
# - report generation  
#####  
  
## Action TAF should perform.  
# Run "perl taf.pl --list-actions" for the full list.  
taf.action=init-start-db-run-tests  
  
# Optional comment describing the purpose of this run.  
taf.comments="MariaDB Cached Reads performance using Sysbench-lua test suite"  
  
# Host on which the test is executed.  
taf.host=localhost  
  
#####  
# TEST EXECUTION SETTINGS  
#####  
  
# Duration of the workload in seconds.  
taf.duration=300  
  
# Number of iterations to run for each thread count.  
taf.iterations=3  
  
# Name of the test suite to load.  
taf.test_suite=sysbench-lua  
  
# Classification of this run.  
# Valid values: adhoc, investigation, production, rerun, release  
taf.test_type=adhoc  
  
# Comma-separated list of test cases to execute.  
taf.tests=POINT_SELECT,OLTP_RO,OLTP_RW  
  
# Comma-separated list of thread counts to test.  
taf.threads=4,8,16,32,64,128
```

```

# Warmup duration in seconds.
taf.warmup_duration=200

# Number of warmup threads.
taf.warmup_threads=64

#####
# TIMING CONTROLS
#####

# Sleep X seconds after test setup.
taf.sleep_after_test_setup=60

# Sleep X seconds after test run.
taf.sleep_after_test_run=60

#####
# REPORT GENERATION
#####

# Comma-separated list of report plugins to run after the test.
taf.report_plugin=taf_res_raw_text.pm,text.pm,chart_and_test_info_results_tables_html.pm

#####
# DATABASE SETTINGS
#
# These settings control how TAF connects to the MySQL server for this run.
# Any value left commented out will fall back to:
# 1. TAF defaults
# 2. auto-populated values (special null handling)
# 3. command-line overrides
#####
# Whether to connect using a UNIX socket instead of TCP.
# true = use socket
# false = use TCP (default)
taf.db_clients_use_unix_socket=true

# Path to the MariaDB configuration file to feed into mariadb.
# This must be a valid MariaDB config file.
taf.db_config_file=./database_config_files/mariadb/mariadb_simple_2gbp.cnf

# Optional: directory where the database software has been unpacked.
# If omitted, TAF attempts to auto-detect the install directory.
# taf.db_software_install_dir=/home/jeb/taf-perl/database_software_installs/mariadb-12.3.0-linux-systemd-x86_64/

# Optional: explicit socket path for MySQL connections.
# Only needed when using UNIX socket mode.
# taf.db_socket=/home/jeb/taf-perl/tmp/mariadb.sock

# Optional: override the storage engine.
# Default is taken from TAF or test suite defaults.
taf.db_engine=innodb

```

```
#####
# SYSBENCH-LUA SETTINGS
#
# These settings apply ONLY to the sysbench-lua test suite. They control:
# - whether BMK integration is enabled
# - table and row counts for workload generation
# - intermediate result reporting
#
# NOTE:
# If sysbench_lua.use_bmk is commented out or set to false, TAF will
# attempt to run the native sysbench-lua executable instead, provided
# it has been built and is available under client_source/sysbench-lua/.
#
# These values override the defaults in sysbench_lua default.properties.
#####

# Which db driver
sysbench_lua.db_driver=mariadb

# Enable BMK integration for sysbench-lua.
# Comment this out or set to false to use the native sysbench-lua executable.
#sysbench_lua.use_bmk=true

# Number of rows per table for sysbench workloads.
sysbench_lua.number_of_rows=1000000

# Number of tables to create for the workload.
sysbench_lua.number_of_tables=8

# Intermediate result reporting mode.
# 0 = disabled
# 1 = minimal
# 2 = detailed
sysbench_lua.intermediate_result=2
#####
# SYSBENCH DEBUG SETTINGS
#####

# Enable verbose sysbench debugging output.
sysbench_lua.debug_sysbench=false

# TAF Debug
taf.tools_debug=true
taf.verbose=true
```

End of Document

This Quick Start was prepared and published by the MariaDB Foundation <https://mariadb.org>

TAF::Perl is part of the Foundation's commitment to open, vendor-neutral tooling for the database ecosystem. The framework is maintained to support fair benchmarking, reproducible testing, and transparent engineering practices across all database makers.

For contributions, feedback, or questions, please visit the Foundation website or contact the maintainers directly.

© MariaDB Foundation. All rights reserved.